

Algorithms and Data Structures

Durham University - COMP1081

MATTHEW EMERSON

April 2026

Contents

I	Fundamental Data Structures	1
1	Introduction	1
1.1	Basic Definitions	1
1.2	Recursion and Backtracking	1
2	Basic Data Structures	2
2.1	Arrays	2
2.2	Lists	2
2.3	Stacks	2
2.4	Queues	3
2.5	Hash Tables	3
II	Asymptotic Notation and Sorting	4
3	Asymptotic Notation	4
3.1	Types of Complexity	4
3.2	General Notation	4
4	Sorting	5
4.1	Common Algorithms	5
4.2	Recursion for Sorting and Solving Recurrences	6
III	Selection and Data Structures	7
5	Algorithms for Sorting, Searching, and Selection	7
5.1	Sorting	7
5.2	Searching	7
5.3	Selection	7
6	Trees	7
6.1	Binary Search Trees	7
6.2	Balanced Binary Search Trees	8
6.3	AVL Trees	9
7	Heaps	9
7.1	Heap Properties	9
7.2	Heap Operations	10
8	Lower Bounds for Sorting and Selection	11
IV	Graph Theory and Algorithms	12
V	References	13
9	Module Professors	13
10	Cited Works	13

Part I

Fundamental Data Structures

1 Introduction

1.1 Basic Definitions

Definition 1.1.1: An algorithm is a method or process that is followed to solve a problem. It must be correct, must be composed of a finite number of unambiguous steps, and must terminate.

Definition 1.1.2: A data structure is a particular way of storing and organising data in a computer so that it can be used efficiently.

Definition 1.1.3: The RAM Model of Computation is a highly simplified model of computation that we use to compute the efficiency of an algorithm. Its memory consists of an infinite array which it uses to carry out instructions sequentially. All instructions take unit time, meaning the running time is the number of instructions executed.

Definition 1.1.4: Pseudocode is a generic way of describing how an algorithm will function without choosing any specific programming language. It does not have any very strict rules, but follows the following generic format:

Algorithm 1 Example pseudocode

Input: Input conditions

Output: The algorithm's output

```
 $x \leftarrow 1$ 
 $y \leftarrow 2$ 
if  $y > x$  then
    return  $y$ 
else
    return  $x$ 
end if
```

Definition 1.1.5: A variable is used to store some value, for example, in the above code, x and y are variables. Some basic types include integer, float, char, and string.

1.2 Recursion and Backtracking

Definition 1.2.1: A recursive algorithm is an algorithm which calls itself to do a part of its work. It should have a base case (where the final result is returned), must change its state to move towards the base case, and must call itself recursively.

Definition 1.2.2: A backtracking algorithm tries to construct a solution to a computational problem incrementally, one small piece at a time. Whenever the algorithm needs to decide between multiple alternatives to the next component of the solution, it recursively evaluates every alternative and then chooses the best one. Definition from (Erickson, 2019, p.71).

Definition 1.2.3: Memoisation is the process of storing the result of a computation so that it can be subsequently retrieved without repeating the computation.

2 Basic Data Structures

2.1 Arrays

Definition 2.1.1: An array is a sequence of elements that is usually denoted as $A[1]$, $A[2]$, ..., $A[n]$ or as $A[1, \dots, n]$. The elements are stored in consecutive memory cells and are all of the same data type.

2.2 Lists

Lists can be separated into three different categories, depending on how they store data.

Definition 2.2.1: A singly-linked list consists of nodes, each storing an element and a pointer to the subsequent node in the list. This means that the nodes do not have to be stored consecutively.

- **Definition 2.2.2:** The first node in a singly-linked list is called the head.
- **Definition 2.2.3:** The final node in a singly-linked list is called the tail and its pointer points to null.

Definition 2.2.4: A doubly-linked list consists of nodes with a value and two pointers, pointing to both the previous and subsequent nodes in the list. The list also must keep a size counter to track the number of nodes in the list.

- **Definition 2.2.5:** At either end of the doubly-linked list is a sentinel node. These nodes either have a null next or previous pointer and do not store a value. An empty list has two sentinel nodes pointing at each other.

Definition 2.2.6: A circularly-linked list uses the same nodes as a singly-linked list but does not have a head or tail node. Instead, the final node points to the start of the list, creating a circle.

- **Definition 2.2.7:** One node in the circularly-linked list is designated as the cursor and is used as a starting point when traversing the list.

2.3 Stacks

Definition 2.3.1: A stack is a collection of objects that are inserted and removed according to the last-in-first-out (LIFO) principle. They support the following methods:

- **Compulsory Methods:**
 - **Push:** $push(element)$ inserts $element$ at the top of the stack.
 - **Pop:** $pop()$ removes and returns the top element from the stack. An error occurs if the stack is empty.
- **Optional Methods:**
 - **Size:** $size()$ returns the number of elements in the stack.
 - **Is Empty:** $isEmpty()$ returns true if the stack is empty, and false if not.
 - **Top:** $top()$ returns the top item from the stack without removing it. Throws an error if the stack is empty.

Stacks are generally implemented using an array to store the items of the stack; however this can lead to memory wastage when the stack is not full, and conversely it will throw errors if the stack is full.

2.4 Queues

Definition 2.4.1: A queue is a collection of objects that are inserted and removed according to the first-in-first-out (FIFO) principle. They support the following methods:

- **Compulsory Methods:**

- **Enqueue:** *enqueue(element)* inserts *element* at the end of the queue.
- **Dequeue:** *dequeue()* removes the element from the front of the queue. An error occurs if the queue is empty.

- **Optional Methods:**

- **Size:** *size()* returns the number of elements in the queue.
- **Is Empty:** *isEmpty()* returns true if the queue is empty, and false if not.
- **Front:** *front()* returns the element from the front of the queue without removing it. Throws an error if the queue is empty.

2.5 Hash Tables

Definition 2.5.1: A hash table consists of a bucket array and a hash function.

Definition 2.5.2: A bucket array for a hash table is an array A of size N , where each cell of A is thought of as a bucket storing a collection of key-value pairs.

Definition 2.5.3: The size N of the array is called the capacity of the hash table.

Definition 2.5.4: A hash function h is a function mapping each key k to an integer in the range $[0, N - 1]$, where N is the capacity of the hash table. This provides an index $h(k)$ of a location in A where the key-value pair (k, v) is stored. Hash functions are usually composed of the hash code (mapping keys to integers) and a compression function (integers to indices).

Definition 2.5.5: A collision occurs when two keys map to the same hash value, and consequently want to be stored in the same bucket. These can be dealt with using different methods:

- **Definition 2.5.6:** Separate chaining converts each bucket to a list of key-value pairs. This will result in lists of average length $\frac{N}{M}$ where N is the amount of data, and M is the capacity of the hash table.
- **Definition 2.5.7:** If a collision occurs at $h(k) = i$, linear probing tries to insert again at $A[(i + 1) \% N]$, then if that is also full, tries at $A[(i + 2) \% N]$ etc. until the key-value pair is inserted.
- **Definition 2.5.8:** Quadratic probing iteratively tries the buckets $A[(i + f(j)) \% N]$ for $j = 0, 1, 2, \dots$ where $f(j) = j^2$ until finding an empty bucket. This avoids clustering patterns that occur with linear probing. However, it creates secondary clustering. This strategy may not find an empty bucket in A even if one exists.
- **Definition 2.5.9:** Using double hashing, we choose a secondary hash function h' , and if h maps some key k to a bucket $A[i]$, with $i = h(k)$, that is already occupied, then we iteratively try the buckets $A[(i + f(j)) \% N]$ for $j = 0, 1, 2, \dots$, where $f(j) = j \cdot h'(k)$. A common choice of secondary hash function is $h'(k) = q - (k \% q)$, for some prime number $q < N$. The secondary hash function should not evaluate to 0.

Definition 2.5.10: Deletions from the hash table must not hinder future searches. To ensure future searches are not hindered, we use tombstones. These are markers that are left in a bucket after a deletion. If a tombstone is encountered when searching along a probe sequence to find a key, we know to continue. The use of tombstones lengthens the average probe sequence distance. Two possible remedies are:

- **Definition 2.5.11:** Using local reorganisation, after deleting a key, we continue to follow the probe sequence of that key and move records into the vacated bucket.
- Or, we can periodically rehash the table by reinserting all records into a new hash table. If you have a record of which keys are accessed most, these can be placed where they can be found most easily.

Part II

Asymptotic Notation and Sorting

3 Asymptotic Notation

3.1 Types of Complexity

Definition 3.1.1: The time complexity of an algorithm can be expressed as the number of basic operations executed by an algorithm when computing for a set input size.

Definition 3.1.2: The space complexity of an algorithm is the amount of memory required to carry out the algorithm on an input of a set size.

Definition 3.1.3: The worst-case time complexity of an algorithm is the largest number of basic operations possible for an input of a set size.

3.2 General Notation

Definition 3.2.1: Generally, Big-O Notation denotes that, if $f = O(g)$, then f will grow no faster than g (the upper-bound for the growth rate, or $f' \leq g'$). Formally, let f be a real or complex function, and let g be a real function such that

$$f(x) = O(g(x)) \quad \text{as } x \rightarrow \infty \quad (1)$$

meaning $f(x)$ is Big-O $g(x)$ if the absolute value of $f(x)$ is at most a positive constant multiple of the absolute value of $g(x)$ for a suitably large value of x :

$$|f(x)| \leq C \cdot |g(x)| \quad x \geq x_0, \quad x_0 \in \mathbb{R}, \quad C \in \mathbb{N} \quad (2)$$

Definition 3.2.2: Similarly, Big-Omega Notation denotes that, if $f = \Omega(g)$, then f will grow at least as fast as g (the lower-bound for the growth rate, or $f' \geq g'$). Formally, let f and g be functions $\mathbb{R} \rightarrow \mathbb{R}$. We say that

$$f(x) = \Omega(g(x)) \quad (3)$$

if, for a suitably large value of x , there exists a constant C such that

$$|f(x)| \geq C \cdot |g(x)| \quad x \geq x_0, \quad x_0 \in \mathbb{R}, \quad C \in \mathbb{N} \quad (4)$$

Therefore, $f(x)$ is $\Omega(g(x))$ if $g(x)$ is $O(f(x))$.

Definition 3.2.3: Big-Theta Notation denotes the exact growth rate of a function. This means that, for some real-valued functions f and g ,

$$f(x) = O(g(x)) \quad \text{and} \quad f(x) = \Omega(g(x)) \quad (5)$$

Definition 3.2.4: Little-o Notation can be used for disregarding “small-order” terms. Generally, $f(x) = o(g(x))$ if $f'(x) < g'(x)$. It is formally defined for the real-valued functions f and g such that $f(x) = o(g(x))$ when

$$\lim_{x \rightarrow \infty} \frac{f(x)}{g(x)} = 0 \quad (6)$$

When the limit is removed, this becomes

$$o(g) = \{f : \mathbb{N} \rightarrow \mathbb{N} \mid \forall C > 0 \exists k > 0 : C \cdot f(n) < g(n) \forall n \geq k\} \quad (7)$$

Therefore, $f(x) = o(g(x)) \implies f(x) = O(g(x))$.

Definition 3.2.5: Similar to Little-o Notation, Little-omega Notation is defined such that $f = \omega(g)$ if $f' > g'$, and is defined formally as

$$\omega(g) = \{f : \mathbb{N} \rightarrow \mathbb{N} \mid \forall C > 0 \exists k > 0 : f(n) > C \cdot g(n) \forall n \geq k\} \quad (8)$$

Definition 3.2.6: A function is designated as a sublinear function if it grows slower than any linear function. Using Little-o Notation, $f(x)$ is sublinear if $f(x) = o(x)$, meaning

$$\lim_{x \rightarrow \infty} \frac{f(x)}{x} = 0 \quad (9)$$

4 Sorting

4.1 Common Algorithms

Definition 4.1.1: An insertion sort works by taking the i^{th} element from a list and inserting it into an already sorted sequence of elements $[1, j - 1]$. This has a best-case runtime of $(n - 1)$ and a worst-case of $\frac{n(n-1)}{2}$, giving a Big-O of $O(n^2)$.

Definition 4.1.2: A selection sort takes the i^{th} smallest elements and places them in order at the start of the list until all elements have been sorted. Its time complexity can be calculated by doing the following:

$$\sum_{i=1}^{n-1} \sum_{j=i+1}^n 1 = \sum_{i=1}^{n-1} (n - i) = \left(\sum_{i=1}^{n-1} n \right) - \left(\sum_{i=1}^{n-1} i \right) = n(n - 1) - \frac{n(n - 1)}{2} = O(n^2)$$

Definition 4.1.3: A bubble sort takes the i^{th} largest elements and places them in order at the end of the list until all elements have been sorted. Its time complexity can be calculated by doing the following:

$$\sum_{i=1}^{n-1} (n - i) = O(n^2)$$

Definition 4.1.4: A merge sort takes a list of elements and recursively splits it into two even parts until each sublist contains one element. Once this occurs, the sublists are merged back together in the correct order to form the sorted list. It does this with a variant of an insertion sort. It has a time complexity of $O(n \log n)$.

Definition 4.1.5: A quick sort takes the first, middle, and final element of a list and selects one as a pivot point. The remaining elements are then placed in sublists depending on whether they are smaller/larger than the pivot. This gives a best-case time complexity of $o(n \log n)$ and a worst-case of $O(n^2)$.

4.2 Recursion for Sorting and Solving Recurrences

To analyse recursive algorithms, first of all note that most of them are actually hybrids between iterative and strictly recursive. For example, merge sort splits the input of size n into two halves of equal size $\frac{n}{2}$, independently recurses in each half, then merges the resulting sorted sequence.

$$T(n) \leq \begin{cases} d & \text{if } n \leq c \text{ for constants } c, d > 0 \\ 2 \cdot T(\frac{n}{2}) + a \cdot n & \text{otherwise} \end{cases} \quad (10)$$

Definition 4.2.1: The Master Theorem can be used to quickly solve recurrences of form

$$T(n) \leq aT\left(\frac{n}{b}\right) + f(n) \quad a \geq 1, b > 1 \quad (11)$$

The theorem has three cases:

- $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0 \implies T(n) = \Theta(n^{\log_b a})$
- $f(n) = \Theta(n^{\log_b a} \cdot (\log n)^k)$ $\implies T(n) = \Theta(n^{\log_b a} \cdot (\log n)^{k+1})$
- $f(n) = O(n^{\log_b a + \epsilon})$, $\epsilon > 0$ and $af(\frac{n}{b}) \leq cf(n)$, $c < 1 \implies T(n) = \Theta(f(n))$

Part III

Selection and Data Structures

5 Algorithms for Sorting, Searching, and Selection

5.1 Sorting

Definition 5.1.1: A bucket sort algorithm creates buckets with labels in the interval $[1, k]$ where k is the largest value in the list. Each element is then placed in its corresponding bucket before the buckets are outputted in ascending order, creating a sorted list. This gives a worst-case time complexity of $O(n + k)$. If k is small relative to n , this gives a best-case time complexity of $o(n \log n)$.

Definition 5.1.2: A bucket sort recursively sorts a list of items by a given digit of each element until the list is sorted (from right-most digit to left-most). This algorithm requires as many buckets as the base of the number system being used (e.g. binary requires two buckets). It has a time complexity of $O(n \log_d k)$ for a list of values in $[1, k - 1]$ in base d .

5.2 Searching

Definition 5.2.1: A binary search assumes that the list being searched is sorted. It recursively halves the search space on each pass until the desired element is found. This produces the recurrence

$$T(n) = T\left(\frac{n}{2}\right) + O(1) = O(\log n) \quad (12)$$

5.3 Selection

Selection algorithms are used to, for example, find the i^{th} smallest element in an unsorted list.

Definition 5.3.1: The quick select algorithm works similarly to the quick sort, using pivots to split the search space and reduce execution times. In the worst-case, the algorithm is $O(n^2)$, but on average executes in $O(n)$ time.

Definition 5.3.2: The median-of-medians algorithm is doubly recursive and works by calculating the median of the list and using it as a pivot. This gives the recurrence

$$T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10} + 3\right) + n = O(n) \quad (13)$$

6 Trees

6.1 Binary Search Trees

Definition 6.1.1: A rooted binary tree is a tree where each node has a single parent node and at most two children nodes. The binary tree is rooted if there is one node with all other nodes as descendants.

Tree Properties:

- **Definition 6.1.2:** A tree's size is the total number of nodes in the tree.
- **Definition 6.1.3:** A node's depth is its distance from the root node.

Definition 6.1.4: A binary tree becomes a binary search tree if, for all nodes, all left-descendant nodes are smaller than it, and all right-descendant nodes are greater than it.

Theorem 6.1.1: Let T be a rooted binary tree of height h . Then,

- T has at most $2^{h+1} - 1$ nodes.
- T has at most 2^h leaves.

Proof: The maximum number of nodes is in a complete tree of height h , meaning all levels are full,

$$1 + 2 + 2^2 + 2^3 + \dots + 2^h = \sum_{n=0}^h 2^n = 2^{h+1} - 1 \quad (14)$$

The second statement follows by induction on h . Case $h = 0$ is obvious. We then consider the left and right subtrees of the root of T . Then each of them has height $\leq (h - 1)$, and (by induction hypothesis) $\leq 2^{h-1}$ leaves. Then T has at most $2 \times 2^{h-1} = 2^h$ leaves.

□

Trees can be traversed in one of three ways, these are:

- **Definition 6.1.5:** Preorder traversal where the root is visited, followed by the left and right subtrees.
- **Definition 6.1.6:** Inorder traversal where the root is visited in between the left and right subtrees.
- **Definition 6.1.7:** Postorder traversal where the root is visited after both the left and right subtrees.

6.2 Balanced Binary Search Trees

Definition 6.2.1: Balanced binary search trees are binary search trees which balance themselves using rotation operations to minimise height differences between subtrees.

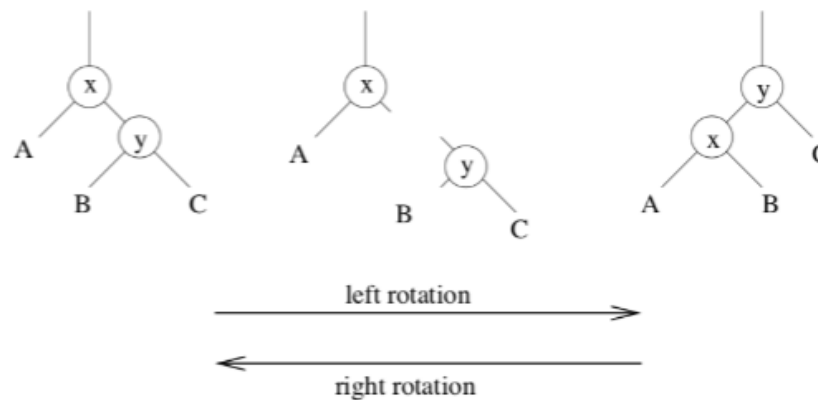


Figure 1: The basic rotation operations.

6.3 AVL Trees

Definition 6.3.1: An AVL tree is a self-balancing binary search tree with the height-balance property (for each node, the height of its children differ by at most one).

Theorem 6.3.1: The height of an AVL tree storing n nodes is $O(\log n)$

Proof: Let us bound $n(h)$ as the minimum number of internal nodes of an AVL tree of height h . We can clearly see that $n(1) = 1$ and $n(2) = 2$. For $n > 2$, an AVL tree of height h contains the root node, one AVL subtree of height $n(h-1)$, and another of height $n(h-2)$. That is,

$$n(h) = n(h-1) + n(h-2) \quad (15)$$

Knowing that $n(h-1) > n(h-2)$, we get $n(h) > 2n(h-2)$ so, by induction,

$$n(h) > 2^i n(h-2i) \quad (16)$$

Taking i to be $\frac{h}{2} - 1$, so that $h - 2i = 1$ or 2 , on solving we get,

$$n(h) > 2^{\frac{h}{2}-1} \quad (17)$$

Taking logarithms, $h < 2\log(n(h)) + 2$. Since $\log(n) > \log(n(h))$, the height of an AVL tree is $O(\log n)$. □

Definition 6.3.2: The algorithm for AVL insertion automatically rebalances the tree after inserting a new element to uphold the height-balance property. For example, for a new node n ,

- If it was inserted into the left subtree of a left child, the algorithm performs a single right rotation.
- If it was inserted into the right subtree of a right child, the algorithm performs a single left rotation.
- If it was inserted into the right subtree of a left child, the algorithm performs a left rotation followed by a right rotation.
- If it was inserted into the left subtree of a right child, the algorithm performs a right rotation followed by a left rotation.

7 Heaps

7.1 Heap Properties

Definition 7.1.1: A priority queue is a collection of prioritised elements that allows arbitrary element insertion and removal having the first priority (highest or lowest).

Definition 7.1.2: All heaps are complete binary trees. The first item in a heap is the root, the second is the left child, and the third is the right child. The next nodes always fill the next levels left-to-right.

Definition 7.1.3: A max heap stores values where, for every node except the root, the value is at most that of its parent. In any subtree, no values are larger than the value stored in the subtree root.

Definition 7.1.4: A min heap stores values where, for every node except the root, the value is at least that of its parent. In any subtree, no values are smaller than the value stored in the subtree root.

Definition 7.1.5: Assuming that we start counting at one for the root, the heap indices can be calculated such that, for a node at index i , the left child is at $2i$, the right child is at $2i + 1$, and the parent is at $\lfloor \frac{i}{2} \rfloor$.

7.2 Heap Operations

Definition 7.2.1: Heapify takes an incorrectly ordered heap and reorders it into an acceptable order. It has a runtime of $O(\log n)$.

Algorithm 2 The Heapify algorithm

Input: A , the array to turn into a heap, v , the index of the largest value, i , the highest index.

Output: A correctly ordered heap.

largest $\leftarrow v$

leftChild $\leftarrow 2v$

rightChild $\leftarrow 2v + 1$

if leftChild $\leq i$ and $A[\text{leftChild}] > A[\text{largest}]$ **then**

 largest $\leftarrow \text{leftChild}$

end if

if rightChild $\leq i$ and $A[\text{rightChild}] > A[\text{largest}]$ **then**

 largest $\leftarrow \text{rightChild}$

end if

if largest $\neq v$ **then**

 Swap($A[v]$, $A[\text{largest}]$)

 Heapify(A , largest, i)

end if

Definition 7.2.2: Insertion adds the new element to the bottom-left of the heap before running Heapify.

Definition 7.2.3: Build heap takes an array of numbers and converts it into a heap by running Heapify iteratively, starting v as the size of the array and looping down to one.

Theorem 7.2.4: The build heap algorithm runs in $O(n)$ time.

Proof: Let $T(n)$ be the execution time for a heap of size n .

$$T(n) = \sum_{h=1}^{\log n} (\# \text{ of nodes at height } h) \cdot O(h) \quad (18)$$

Given the properties of binary trees,

$$T(n) \leq \sum_{h=1}^{\log n} \frac{n}{2^h} \cdot O(h) = O\left(n \cdot \sum_{h=1}^{\log n} \frac{h}{2^h}\right) \quad (19)$$

This is well known for $x \in \{0, 1\}$,

$$\sum_{i=0}^{\infty} i \cdot x^i = \frac{x}{(1-x)^2} \quad \sum_{h=1}^{\log n} \frac{h}{2^h} \leq \sum_{h=0}^{\infty} h \cdot \left(\frac{1}{2}\right)^h = 2 \quad (20)$$

and hence,

$$T(n) = O\left(n \cdot \sum_{h=1}^{\log n} \frac{h}{2^h}\right) = O(n) \quad (21)$$

□

8 Lower Bounds for Sorting and Selection

Theorem 8.0.1: Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Proof: It is sufficient to determine the minimum height of a decision tree in which each permutation appears as a leaf. We can consider a decision tree of height h with l leaves corresponding to a comparison sort on n elements. Each of the $n!$ permutations of the input appears as some leaf, $l \geq n!$. Given that a binary tree of height h has at most 2^h leaves, we get that

$$n! \leq l \leq 2^h \quad (22)$$

Therefore, $n! \leq 2^h$, and after taking logs of both sides,

$$h \geq \log(n!) \equiv \Omega(n \log n) \quad (23)$$

□

Definition 8.0.1: An adversary algorithm is designed to intercept and limit access to the input by the original algorithm such that the time taken to make a decision is maximised. However, the adversary does not have knowledge of how the algorithm works, so must be general enough to work for any algorithm provided.

Theorem 8.0.2: Any comparison-based algorithm attempting to find the maximum element in an unsorted array requires at least $(n - 1)$ comparisons in the worst case.

Proof: After at most $(n - 2)$ comparisons no more than two elements can have not lost a comparison. An adversary can make either of them max and be consistent, but there is not enough information for the original algorithm to be consistent. Hence, the algorithm needs to make one more comparison, making the minimum number of comparisons $(n - 1)$.

□

Alternative Proof: Taking this into account, we can design a strategy for the adversary. The only relevant information for the algorithm is how many elements have lost at least one comparison. As soon as $(n - 1)$ elements have lost, the algorithm can decide that the only undefeated element must be the maximum.

Assuming each index i has a status (N for never lost, and L for lost), initially all indices have N . After each comparison there can be a status change $N \rightarrow L$. Therefore, the adversary must delay changes $N \rightarrow L$ as much as possible.

Given this information, each comparison will either update one status from N to L , or will update neither, providing the algorithm with at most one bit of new information. In total, the algorithm needs $(n - 1)$ bits of information to make a decision, and hence at least $(n - 1)$ comparisons are necessary.

□

Part IV

Graph Theory and Algorithms

Part V

References

9 Module Professors

The below is accurate as of the 2025/26 academic year and will not be updated in subsequent years.

- *Fundamental Data Structures* and *Graph Theory and Algorithms* are taught by David Kutner.
 - **Office Location:** Room 2011, MCS Building
 - **Website:** <https://www.durham.ac.uk/staff/david-c-kutner/>
 - **Preferred Contact Methods:** david.c.kutner@durham.ac.uk
 - **Pronouns:** he, him
- *Asymptotic Notation and Sorting* is taught by Thomas Erlebach.
 - **Office Location:** Room 2004, MCS Building
 - **Website:** <https://www.durham.ac.uk/staff/thomas-erlebach/>
 - **Preferred Contact Methods:** thomas.erlebach@durham.ac.uk
 - **Pronouns:** he, him
- *Selection and Data Structures* is taught by Anish Jindal.
 - **Office Location:** Room 2065, MCS Building
 - **Website:** <https://www.durham.ac.uk/staff/anish-jindal/>
 - **Preferred Contact Methods:** anish.jindal@durham.ac.uk
 - **Pronouns:** he, him

10 Cited Works

Erickson, J. (2019). Algorithms. 1st ed. [online] Self-Published, p.71.
Available at: <http://jeffe.cs.illinois.edu/teaching/algorithms/> [Accessed 25 Mar. 2026].